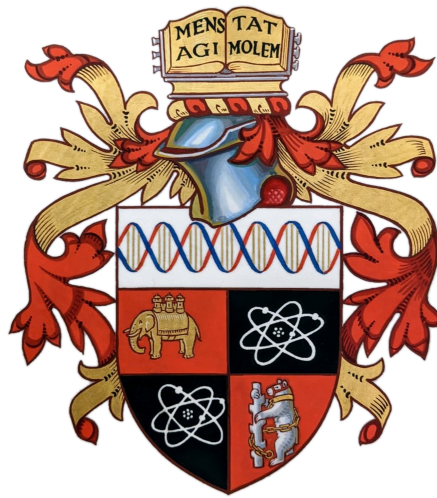




ENGINEERING AND EVALUATING MEMORY RETRIEVAL FOR LLM-DRIVEN BOSS DIALOGUE IN A TURN-BASED GAME

by

SHUNYU YANG

A thesis submitted in partial fulfilment
of the requirements for the degree of

MASTER OF SCIENCE

UNIVERSITY OF WARWICK

WARWICK MANUFACTURING GROUP

August 2026

CONTENTS

Contents	ii
Declaration	v
Abstract	vi
List of abbreviations	vii
1 Introduction	1
1.1 Motivation	1
1.2 Research Problem and Gap	2
1.3 Research Aim and Objectives	3
1.4 Research Question	3
1.5 Contributions	4
1.6 Dissertation Structure	4
2 Related Work	5
2.1 Traditional NPC Dialogue	5
2.2 LLM-generated Dialogue in Games	6
2.3 Memory in Conversational Agents	7
2.4 Retrieval-Augmented Generation	8
2.5 Semantic Similarity and Dense Retrieval	8
2.6 Importance-aware Memory Retrieval	9
2.7 Rule-based and Model-assisted Ranking	10
2.8 Evaluations of Retrieval and Generated Dialogue	10
2.9 Literature Summary and Research Gap	11
3 Research Methodology	13
3.1 Objectives	13
3.2 Research Approach	13
3.3 Study Plan and Outputs	15
3.3.1 Stage 1: Prototype	15
3.3.2 Stage 2: Offline Retrieval	15
3.3.3 Stage 3: Player Study	15

3.3.4	Stage 4: Analysis	15
3.4	Measures of Success	16
4	Method	17
4.1	Prototype Implementation	17
4.2	Gameplay and Experimental Scenario	18
4.3	Dialogue Memory Architecture	20
4.4	Retrieval Strategies	21
4.4.1	Similarity-only Retrieval	21
4.4.2	Rule-based Importance Retrieval	21
4.4.3	Model-assisted Importance Retrieval	22
4.5	Dialogue Generation	23
4.6	Experiment Procedure and Data Collection	23
5	Results	25
5.1	Dataset	25
5.2	Offline Retrieval Evaluation	25
5.3	Questionnaire Results	27
5.4	Latency	29
6	Analysis	30
6.1	Offline Retrieval	30
6.2	Player Experience	31
6.3	Engineering Interpretation	31
6.4	Answer to the RQ	32
7	Discussion	34
7.1	Summary of Analysis	34
7.2	Implications for Game Developers	34
7.2.1	Implications for Game Designer	34
7.2.2	Implications for Game Engineering	35
7.3	Implications for Researchers	36
7.4	Limitations	37
7.4.1	Test Environment and Participants	37
7.4.2	Control of the Playable Study	37
7.4.3	Technical Generalisability	37
7.5	Future Work	38
8	Conclusion	40
8.1	Study Summary	40
8.2	Main Findings	40
8.3	Contributions	41

A	Appendices	42
A.1	Participant Information, Consent, and Questionnaire	42
A.1.1	Participant Information and Consent	42
A.1.2	Qualtrics Presentation Flow	42
A.1.3	Repeated Post-condition Questionnaire	43
A.1.4	Personality, Comparative, and Open Questions	43
A.2	Dialogue and Importance-scoring Prompt Specification	44
A.2.1	Dialogue-generation Prompt	44
A.3	Memory Schema	45
A.4	Human Relevance Labeling Guide	46
A.5	Experimental Condition Orders	46
A.6	Release, Proxy Configuration, and Reproducibility	47
	Bibliography	49

DECLARATION

This thesis is submitted to the University of Warwick in support of my application for the degree of Master of Science. It has been composed by myself and has not been submitted in any previous application for any degree. The work presented (including data generated and data analysis) was carried out by the author.

ABSTRACT

Large language model (LLM)-generated NPC dialogue requires a game to record player-history events and select suitable memories for each dialogue request. This study addresses the practical problem of how this selection can be managed within a small game project. It implemented and compared three retrieval strategies in a Unity prototype: Similarity-only, Rule-based Importance, and Model-assisted Importance. The strategies were evaluated through a fixed offline benchmark, an eleven-participant playable study, and runtime logs. Similarity-only retrieved the largest number of at least relevant memories, Model-assisted produced the strongest graded ordering, and Rule-based offered the clearest designer control. Although Model-assisted Importance produced the strongest graded offline ranking, participants showed no significant perception differences between the strategies. In this pilot, this suggests that better memory ranking does not automatically lead to a detectable improvement in perceived dialogue experience. Runtime evidence also showed that Model-assisted added pre-scoring, caching, and failure-handling requirements, although online retrieval remained very short compared with dialogue generation. The study therefore provides a small-scale, traceable pipeline in which request-linked logs connect retrieval decisions, generated dialogue, latency, and failures, together with practical guidance for choosing a retrieval strategy according to query needs, designer control, and engineering cost.

LIST OF ABBREVIATIONS

API	application programming interface
JSONL	JSON Lines
LLM	large language model
nDCG	normalised discounted cumulative gain
NPC	non-player character
RAG	retrieval-augmented generation
RQ	research question

1 INTRODUCTION

1.1 MOTIVATION

This study began with a simple problem that the author has often encountered as both a player and developer: non-player character (NPC) dialogue can be exciting, but it can also become disappointing very quickly. A game can build a vivid world through visuals, rules, and interaction, but limited time, labour, and cost mean that it cannot respond to every action as the real world might. When a character suddenly stops responding, repeats the same line, or ignores what has just happened, the player can easily feel that they have reached the edge of the game, which breaks the immersion immediately. This problem has been troubling people involved in game development for a long time, and there is still no good solution.

Nevertheless, when time and budget are limited, text and dialogue remain one of the effective way to make the game world feel extensive. The dialogue itself is very powerful; it can directly or indirectly explain the story background, describe and highlight the characters, and even give players room for imagination, allowing players expand the game world themselves [1]. Compared with producing a new scene, animation, or voiced lines for every situation, writing can describe characters, places, and events outside the screen at a lower cost. Despite the emergence of various narrative approaches, the vast majority of games still rely on text to convey the majority of their plots. Interactive narrative games are a prime example of this feature being taken to its extreme.

The problem is that traditional dialogue can only cover a limited number of situations. Most lines are written in advance and triggered by fixed conditions or dialogue trees. This means that developers can check every line, but they must also guess what the player might do beforehand. Once choices, battle results, relationships, and story events begin to combine, the number of cases grows quickly. A team's workload is always limited, so it is inevitable that players encounter missing, repeated, or inconsistent dialogue. Scripts give developers control, but they make it difficult to respond to every meaningful combination of events [2, 3].

In recent years, LLMs have offered another way to address this problem. An LLM can understand text input and create a new response while the game is running [4, 5]. Instead of choosing only from pre-written lines, a game can give the model a character persona, world background, current scene, and records of the player's earlier behaviour, then ask it to form a reply. This will not replace writers or game designers, but it may reduce the need

to write a separate branch for every combination of actions and allow characters to respond to cases that were not prepared line by line.

This idea has already appeared in research prototypes, technical demonstrations, and a small number of commercial games. NVIDIA ACE shows how speech recognition, language generation, voice, and facial animation can be connected in a real-time character system. Related features have also appeared in *inZOI* and a specific version of *NARAKA: BLADEPOINT* [6, 7]. However, these examples also show that the technology is still at an early stage. Some other researches also emphasize that safe content generation have to consider many factors [8, 9]. Besides, an *inZOI* developer update also notes that Smart Zoi sometimes failed to respond correctly as the game gained more content [10]. Successfully generating a line inside a game is therefore only the first step; it does not mean that the whole system is already stable or ready for common use.

These problems are even more challenging for small teams, novice AI developers, or independent game developers. LLM itself is a relatively new concept, and the level of expertise in this area is still catching up, with even fewer people specifically dedicated to game development. Large companies have ample budgets, while small developers urgently need more practical solutions. For example, a project by Arm and North Carolina State University took a direct approach: they customized an efficient local model to approximate the game’s frame rate requirements based on the specific needs of a puzzle game and highly adapted it for collaboration with modern game engines [9]. This practice demonstrates the potential for further improvements in cost and performance, but developers still need to carefully select model scale, prompts, and detailed integration methods. The required expertise remains high.

Therefore, this research will first directly utilize existing mature LLMs to minimize integration, understanding, and budget costs. Then, it will explore how to make game dialogue generation by large language models more practically understandable, controllable, constructible, and supervised: selecting which past game events to provide to the model before generating dialogue. This study does not attempt to find a single best approach applicable to all games, but rather compares the trade-offs between different retrieval methods in terms of sorting quality, developer control, system complexity, wait time, and player-perceived effects. The runnable game prototype was used not only to conduct experiments but also to demonstrate how this choice could be integrated into a complete and maintainable game dialogue flow.

1.2 RESEARCH PROBLEM AND GAP

For reliable generated dialogue, the model needs access to the character definition, the current situation, and the player’s earlier actions. A general model does not know the events of one game session, and separate API requests to model do not automatically preserve the player’s experience. The game must therefore record and provide this information again. Supplying the complete history may appear straightforward, but longer context does not

guarantee that every item will be used correctly and may also increase input size and waiting time [11, 12]. The practical problem therefore includes not only the model’s capacity to generate a line, but also the repeatable and inspectable selection of information required by each request.

This narrows the problem to the selection of past events. Too many unrelated records may hide important information or encourage a forced reference, while too few may make the character appear to forget the player. Semantic similarity also does not necessarily show which event matters most to a character. As the result, retrieval must balance between relevance to the current request with the significance of an event and character relationships.

Research on retrieval-augmented generation (RAG) and agent memory shows that external information can be retrieved before generation and ranked through signals such as similarity and importance [13–15]. However, there is limited game-specific evidence directly comparing semantic similarity, designer-defined importance, and model-assisted importance under the same memories, queries, and TopK. Small developers also lack complete playable cases showing how these approaches can be integrated, inspected, and recovered from failure. Together, these gaps define the scope of this study.

1.3 RESEARCH AIM AND OBJECTIVES

This study aims to implement and compare Similarity-only, Rule-based Importance, and Model-assisted Importance within the same small game-dialogue pipeline. It examines their trade-offs in retrieval quality, designer control, inspectability, runtime cost, and system complexity. Rather than identifying one method as best for every game, it provides a game-specific comparison that can be run and reproduced.

To achieve this aim, the study builds a memory-dialogue pipeline with replaceable retrievers and uses focused queries with one clear target as the primary set for comparing Top3 relevance and graded ranking. Broader queries that may require several parts of the player’s history are retained as supplementary analysis. The playable study then explores whether players notice strategy differences and records the practical operation of each implementation.

1.4 RESEARCH QUESTION

Accordingly, the study addresses the following research question (RQ):

Within a fixed benchmark of turn-based game memories and focused, event-driven dialogue queries, how do similarity-only, rule-based importance, and model-assisted importance retrieval differ in Top3 relevance and graded ranking quality?

The playable study and engineering records provide exploratory practical context rather than a second controlled hypothesis test; their design is described in Chapter 3.

1.5 CONTRIBUTIONS

The first contribution is engineering-based. The study provides a working Unity memory-dialogue prototype containing structured gameplay events, atomic long-term memories, replaceable retrieval strategies, cached model-assisted scores, dialogue-request management, and JSON Lines (JSONL) traces connected through shared `RequestId` values. The resulting conditional guidance is summarised in Table 7.1. The prototype is not presented as a production-ready general framework, but as a reproducible and inspectable case showing small developers how ranking gains, player-visible benefit, system complexity, and runtime cost can be considered separately.

The second contribution is experimental. The study provides a fixed offline retrieval benchmark that compares Similarity-only, Rule-based Importance, and Model-assisted Importance using the same memories, queries, TopK, and human relevance labels. Focused queries form the primary comparison, while broad queries are retained as supplementary evidence. The results also provide an early indication that query scope changes how the strategies differ: focused questions expose direct matching and rule decisions clearly, while broader questions gave Model-assisted Importance more room to improve ordering but also allowed more interpretation. An exploratory player study then observes whether offline retrieval differences remain visible after entering the complete game experience.

1.6 DISSERTATION STRUCTURE

The remainder of this dissertation is organised as follows. Chapter 2 reviews related work on traditional NPC dialogue, LLM-driven game dialogue, agent memory, and retrieval. Chapter 3 explains the research approach, experimental design, participants, and evaluation methods. Chapter 4 describes the game prototype, memory system, three retrieval strategies, dialogue-generation pipeline, and data logging. Chapter 5 reports the offline retrieval, questionnaire, dialogue, and performance results, and Chapter 6 analyses those findings. Chapter 7 discusses their meaning, limitations, and possible future work, before Chapter 8 concludes the dissertation.

2 RELATED WORK

Figure 2.1 provides an overview of the reviewed areas and the representative sources discussed in each part of this chapter. The chapter then connects these areas progressively, from authored dialogue and generated dialogue to memory, retrieval, ranking, and evaluation.

Research combining LLM-generated game dialogue, persistent player memory, and controlled memory retrieval, which are relatively new. Much of the game-specific empirical work used in this dissertation appeared from 2023 onwards, and there is not yet one widely accepted approach covering the whole problem [16–18]. The study, therefore, focus on several related areas, including interactive narrative, LLM-generated dialogue, agent memory, information retrieval, ranking, and evaluation.

This chapter reviews these areas in the order in which they become relevant to the project. It begins with traditional game dialogue and its production limits, then considers the opportunities and problems of LLM-generated dialogue. Later sections cover memory, RAG, semantic similarity, importance-based ranking, and the evaluation of retrieval and dialogue. The purpose is not to repeat the motivation for the study, but to summarise the contributions and unresolved issues in existing work and their relevance to the comparison of three retrieval strategies.

2.1 TRADITIONAL NPC DIALOGUE

Traditional NPC dialogue normally relies on pre-written lines, dialogue trees, and triggers connected to game state. Its main advantage is control: developers can check what a character says, when they say it, and whether the statement fits the story. Research on interactive narrative also shows that this control must be balanced against the player’s sense that their actions can affect events. Riedl and Bulitko describe this as a continuing tension between player agency and a coherent authored story [1]. Players may be free to choose, but the game can only respond through states and possibilities that the designers represented beforehand.

Façade shows how much flexibility an authored system can achieve within this limit [2]. It divides an interactive drama into written story beats and dialogue behaviours, then uses a drama manager to select material in response to the player. This makes the scene more responsive than a simple dialogue tree, but it still requires a large amount of prepared content. More importantly, the system selects scripted dramatic material; it does not retrieve player memories or generate new lines in the way a modern LLM system does.

Chen, Nelson, and Mateas use the term *authorial leverage* to describe how much

Traditional NPC Dialogue and Interactive Narrative	LLM-generated Dialogue in Games	Memory in Conversational and Generative Agents	Retrieval-Augmented Generation
Riedl & Bulitko (2013) Mateas & Stern (2005) Chen et al. (2009) Yu & Riedl (2013)	Vaswani et al. (2017) Brown et al. (2020) Akoury et al. (2023) Peng et al. (2024) Buongiorno et al. (2024) Sadeq et al. (2024) Tseng et al. (2024) Hejmadi (2026)	Baddeley & Hitch (1974) Tulving (1972) Sumers et al. (2023) Park et al. (2023) Packer et al. (2023) Wu et al. (2024)	Guu et al. (2020) Lewis et al. (2020) Borgeaud et al. (2022) Liu et al. (2024) Li et al. (2024) Gao et al. (2023)
Semantic Similarity and Dense Retrieval	Importance-aware Memory Retrieval	Rule-based and Model-assisted Ranking	Evaluation of Retrieval and Generated Dialogue
Salton et al. (1975) Reimers & Gurevych (2019) Karpukhin et al. (2020) Thakur et al. (2021)	Park et al. (2023) Farrell et al. (2019) Xu et al. (2025)	Sun et al. (2023) Buongiorno et al. (2024)	Kent et al. (1955) Kekäläinen & Järvelin (2002) Järvelin & Kekäläinen (2002) McDonnell et al. (2016) Saad-Falcon et al. (2024)

FIGURE 2.1: Reviewed research areas and representative sources.

meaningful changes in game an author’s work can produce [3]. Their comparison shows that content cost comes not only from the number of lines, but also from the logic needed to combine story elements. Yu and Riedl add player modelling so that a story can react to predicted choices while leaving those choices available [19]. These studies show that scripted dialogue is not obsolete: it remains reliable and controllable. Its limitation is that responses stay within prepared content, while new combinations of state also increase the logic that must be maintained. LLMs change the production of the final line, but developers still need a maintainable representation of event history, character knowledge boundaries, and past events worthy of a response.

2.2 LLM-GENERATED DIALOGUE IN GAMES

LLMs make it possible for games to generate dialogue from instructions and contextual information rather than selecting only from pre-written lines[4, 5]. For games, this means that one prompt can contain world background, character personality, current state, and player history before asking the model to create a new line. However, producing a fluent sentence does not mean that the sentence is suitable for the game.

Existing studies show a trade-off between narrative control and generative freedom. Akoury, Yang, and Iyyer evaluate generated dialogue using material from *Disco Elysium: The Final Cut* [16]. GPT-4 receives structured game state and is asked to fill in missing

dialogue. Players who familiar with the game generally prefer the original lines, especially for its logic flow, appropriateness, and use of game events. This result does not prove that all generated dialogue is unsuitable for games, though it shows that fluent language itself cannot reproduce the control over context and style found in professional scripted dialogue.

By contrast, Peng et al. study a more open text adventure game in which players can speak freely with GPT-4-driven characters [17]. Their play log contains events and discoveries that were not designed to the initial storytelling, showing that experimental gameplay and unexpected narrative do appear in generated dialogue. Together, they shows a clear trade-off: open generation gives the player more possibilities, but the game also needs stronger context and rules to keep it consistent.

While PANGeA addresses this problem by combining personality, memory, designer configuration, Unity integration, and a validation stage [18]. Its validation module improves the degree of narrative rules across several models. Sadeq et al. find a related problem: a model may use information that a character should know in future [20]. Tseng et al. further separate role-playing from player personalisation [21]: A fixed personality defines who the character is, while player’s actions can change the attitude of character towards the player. Applied to games, this means that dialogue quality depends on both a stable character definition and the appropriate use of player-specific information. This requirement leads to the question of how earlier game events should be stored and selected.

2.3 MEMORY IN CONVERSATIONAL AGENTS

Memory research provides useful terms for separating information needed now from information stored from the past. Baddeley and Hitch’s attention of working memory concerns information kept active for a current task [22], while Tulving distinguishes episodic memory for experienced events from semantic memory for general knowledge [23]. These psychological concepts are not direct software modules, but they help explain why character background, past events, and current battle state should not all be record as the same kind.

Sumers et al. apply a similar separation to language agents in CoALA, treating working memory, long-term memory, actions, and learning as different parts of a wider system [24]. Storing an event, finding it again, and deciding how to respond are also separate operations. This distinction is useful in games because game code can record what the player actually did, while retrieval and generation can be treated as later problems. It also shows why an evaluation of memory retrieval should distinguish whether an event was recorded correctly from whether it was later selected correctly.

Park et al.’s Generative Agents provide an important example of persistent memory [14]. The system stores observations as natural-language records and later reorders them by relevance, recency, and importance. The agents also create reflections and plans. Its ablation study suggests that memory, reflection, and planning contribute to more believable behaviour. This work shows that memory selection does not have to rely on text similarity alone, but it studies a broad social simulation rather than different methods of scoring

importance inside one game scenario.

MemGPT focuses on the limited space available in active model context and moves information between that context and external storage [25]. LongMemEval shows that access to a longer context still leaves models struggling to find facts across sessions, update old information, reason about time, or recognise when information is absent [26]. These results suggest that memory quality depends not only on the volume of stored information, but also on record formation and subsequent selection. For games, stable and inspectable event records are therefore a necessary basis for evaluating later retrieval.

2.4 RETRIEVAL-AUGMENTED GENERATION

The basic idea of RAG is to allow a language model look up information outside its parameters before generating an answer. REALM adds a learned retriever during pre-training, Lewis et al.’s RAG combines searchable external memory with a generator, and RETRO applies retrieval at a much larger corpus scale [13, 27, 28]. These systems mainly address knowledge tasks, but they establish a distinction that is also useful in games: general information already known by the model is different from information produced by one player’s recent actions.

In a game, searchable material need not be a huge collection; it can consist of events formed through one player’s actual play. Such information cannot exist in the model’s original training data, but it can be stored by the game and retrieved before a character speaks. This creates an important distinction: the retriever decides which records become visible to the model, while the generator decides how those records are used in dialogue.

A longer context does not automatically remove the need for selection. Liu et al. find that models do not use every position in a long input evenly, and relevant evidence in the middle can be easier to miss [11]. Li et al. compare long-context models with RAG and show that the outcome depends on the model, task, input length, and available resources: strong long-context models perform better on some tasks, while retrieval can reduce computational cost [12]. This study therefore does not assume that retrieval is always better than providing every memory. It asks how records should be ranked once retrieval is required. Gao et al. also describe retrieval, augmentation, generation, and evaluation as steps with different sources of error [15]. This separation supports treating the retriever as a replaceable component and observing memory selection separately from the final line, rather than assigning every error to the generation model.

2.5 SEMANTIC SIMILARITY AND DENSE RETRIEVAL

Using vectors to compare text predates large language models. Salton, Wong, and Yang’s vector space model translates the target and query texts into vectors and compares them in a vector space [29]. Modern dense retrieval uses learned representations instead of relying solely on word frequencies. Sentence-BERT optimizes translation, encoding, and faster

vector comparison, making it suitable for repeatedly searching a memory pool [30]. The existence of such dual-encoder methods also makes smaller-scale local semantic retrieval a viable option.

Dense Passage Retrieval further improves its performance in open-domain question answering. It encodes questions and passages independently and searches them in a common vector space [31]. This demonstrates that dense semantic search works effectively on multiple datasets, but it doesn't prove that such a simple embedding model can understand the varying importance of narrative content in games. BEIR further shows that dense retrieval performs inconsistently across different domains and doesn't always outperform lexical methods when transferred to tasks different from the training data [32].

Game memories add a more direct problem: semantic similarity is not the same as importance to the character. Several similarly worded choices may fill the TopK results, while a differently worded event involving harm to the boss's subordinate may be left out. The result also depends on how the query is written: a question about combat ability may favour damage records, while a question about attitude may favour choices. Similarity-only retrieval is therefore a clear and common baseline. It can be deployed with a local sentence-embedding model and relatively few additional components, but it gives the designer no direct place to express a character-specific value judgement.

2.6 IMPORTANCE-AWARE MEMORY RETRIEVAL

Importance-aware retrieval adds a signal about the event itself alongside relevance to the current query. Generative Agents is a main example because it combines relevance, recency, and importance when selecting observations, allowing influential events to remain available after time has passed [14]. This shows how multi-signal ranking may preserve events that current wording alone would overlook, while also making it necessary to evaluate what each signal contributes to a particular task.

Research on narrative salience gives another reason to separate importance from similar wording. Farrell, Ware, and Baker examine how links involving characters, time, place, cause, and intention affect how strongly an event stands out in a narrative [33]. Their work does not provide a score table that applies to every game, but it supports the idea that an event's value depends on who was affected, why it happened, and how it connects to later events. A player's treatment of a character's subordinate may therefore matter greatly to that character even when the record shares few words with the current dialogue trigger.

A-MEM explores a more complex approach, allowing memory records to be updated with new information, including descriptions, tags, and connections [34]. This demonstrates that agent memory can be an architecture that updates in real time, rather than simply a list enumeration. However, existing literature still doesn't propose how to calculate importance in a game, as this score may incorporate designer judgments such as character relationships, combat rules, and narrative intent, rather than just a general mathematical attribute. High importance can also push a dramatic event into irrelevant conversations;

therefore, importance cannot completely replace relevance to the current query.

2.7 RULE-BASED AND MODEL-ASSISTED RANKING

First, there's the rule-based ranking method. This method is similar to the script-based rules used in traditional dialogue systems. Rule-based importance can assign stable scores based on combat rounds, remaining lives, player choices, and character relationships, providing the output stability and controllability most needed in the design. However, the threshold for this score must be interpreted according to the specific mechanism, and boundary tests must be used to confirm that the code is consistent with the written rules. It's easy to reproduce, but it may also miss situations that the developers didn't anticipate.

Related research includes RankGPT. It demonstrates that large language models with cue words can also rank text candidates according to natural language descriptions [35]. However, it studies the relevance of candidate content to the query, rather than the importance of game events independent of the query, so it cannot directly prove that the model can reliably complete the same task. It still shows that the model can apply well-defined ranking conditions, but it also introduces cost and consistency issues. PANGEA also proposes this constrained approach when using explicit game rules, rather than letting the model make free judgments when there are no standards [18].

The two approaches offer different kinds of explanation and different maintenance responsibilities. A rule can point to a known field and threshold, making it easy to reproduce and adjust, but the developer must maintain those rules when mechanics or character definitions change. A model can return a natural-language reason and handle cases that were not listed one by one, but the reason does not reveal internal reasoning and the system also gains dependencies on prompting, service calls, and failure handling. Existing work therefore leaves the added value and cost of more flexible judgement uncertain, as well as the differences produced by the two approaches when applied to the same game task.

2.8 EVALUATIONS OF RETRIEVAL AND GENERATED DIALOGUE

Retrieval and final dialogue need to be evaluated separately. Early work by Kent et al. presents precision and recall as complementary retrieval criteria [36]. Precision@K measures how many of the K returned memories are relevant, while Recall@K measures how many of all labelled relevant memories are returned. These are binary relevance measures, meaning that they distinguish only between relevant and irrelevant items; in this study, label 0 is treated as irrelevant, while labels 1 and 2 are both treated as relevant. With a small prompt budget, precision shows whether unrelated memories take up valuable positions, while recall shows whether useful memories were missed. Recall@K cannot reach one when the number of relevant memories is greater than K, so the two measures need to be interpreted together.

Precision and recall do not reward a strongly relevant memory for appearing first rather than third. Kekalainen and Jarvelin argue that evaluation should preserve degrees of relevance instead of reducing every judgement to relevant or irrelevant [37]. Their work on cumulative gain introduces discounted cumulative gain and its normalised version, normalised discounted cumulative gain (nDCG), which give more value to highly relevant items near the top of a ranking [38]. This kind of graded evaluation suits game memories because a record may be unrelated to a trigger, provide partial support, or directly answer the current need.

Human labels are still judgements rather than absolute truth. McDonnell et al. examine the use of short reasons alongside relevance labels [39]. A shared written guide and brief reasons make disputed cases easier to review, and a second assessor can label a shared subset to check whether the guide is being used consistently. Human relevance labels should therefore be treated as reference judgements made under a fixed guide, with remaining disagreement reported rather than hidden.

ARES also separates the relevance of retrieved context from the quality and faithfulness of the final answer [40]. This distinction is equally important for game dialogue because better retrieval ranking does not automatically produce a better player experience; the generation model, scenario length, and dialogue presentation may all change the final effect. Retrieval metrics, generated outputs, player perception, and system-operation data therefore answer different questions. Connecting them can provide a fuller evaluation, but one form of evidence should not be treated as a replacement for another.

2.9 LITERATURE SUMMARY AND RESEARCH GAP

The reviewed work first shows a trade-off between scripted and generated dialogue. Scripted systems are reliable and easy to control, but their responses are limited by prepared content. LLMs can respond to combinations that were not written line by line, but they may misuse game state, break character, or add unsupported information. Clear persona, world context, and validation rules can reduce these risks, but they do not decide which events from a growing player history should be supplied for one line of dialogue.

Agent-memory and RAG research shows that external records can give a model player information that was not available during training. Work on long context also shows that placing every record in the input does not guarantee that the model will use each one correctly. Semantic similarity provides an efficient and clear selection baseline, but retrieval benchmarks and narrative-salience research both suggest that similar wording cannot fully represent how much an event matters to a particular character.

Importance-aware systems offer one possible direction, but existing work usually addresses broader tasks such as social simulation, question answering, or long-term conversation. LLM-ranking studies normally judge candidate relevance to a query rather than storing a separate estimate of an event’s narrative importance to a game character. There is still limited work providing a game-specific benchmark that directly compares semantic

similarity, designer-defined importance, and model-assisted importance with the same memories, predefined queries, and TopK. Small developers also lack reproducible and observable end-to-end cases showing how these methods can become replaceable game components, and whether ranking quality, runtime cost, failure handling, and player experience move in the same direction.

This project addresses that narrower gap using episodic memories formed through actual turn-based play. It does not attempt to prove that retrieval is always better than scripted dialogue or full-context prompting. Once retrieval is required, it compares the three strategies mainly through focused queries with one clear target. Broader queries are retained as supplementary evidence rather than being folded into the main answer:

These limitations lead directly to the research question introduced in Chapter 1: how the three retrieval strategies differ under the same game memories, focused queries, and Top3 setting.

RESEARCH METHODOLOGY



This chapter explains the research design used to address the research question and describes how the different forms of evidence were collected.

3.1 OBJECTIVES

Besides the 1.4: how three practical methods select Top3 memories for one identifiable game event or state once a dialogue system has decided to use retrieval. The study is also trying to provide a comprehensive view of a such system to developers. Therefore, evaluation comes from two separate levels, offline benchmark for detailed performance, questionnaire for player experience.

To do this, the fixed offline benchmark is designed to use same memories, focused queries, Top3, and human labels, in order to answer the main question. While broader queries provide supplementary analysis. The playable study explores whether ranking differences remain visible after entering a complete dialogue system, while system logs describe checkability, runtime cost, and observed failures, which can be further data for both direction.

Figure 3.1 summarises these stages and the evidence produced by each one, showing how the primary offline comparison remains separate from the supporting player and engineering observations.

3.2 RESEARCH APPROACH

Specifically, the offline test is required to remove player behaviour and generation variation to compare memory ranking directly. The playable test need to retain normal combat variation, explores whether ranking differences remain visible in the complete system, and records delay and failure. While Open responses and saved dialogue traces in Qualtrics provide examples rather than exhaustive factual-correctness coding.

When selecting the playable prototype, the study also had to consider the unavoidable waiting time of an external generation model. Turn-based combat is not the only type of game that can accommodate this delay. Menu-based dialogue, management or simulation play, and interactions triggered during natural pauses can also place generation requests where continuous player input is not required. This study selected turn-based card combat because it provides clear points at the start of turns, between actions, and at battle end,

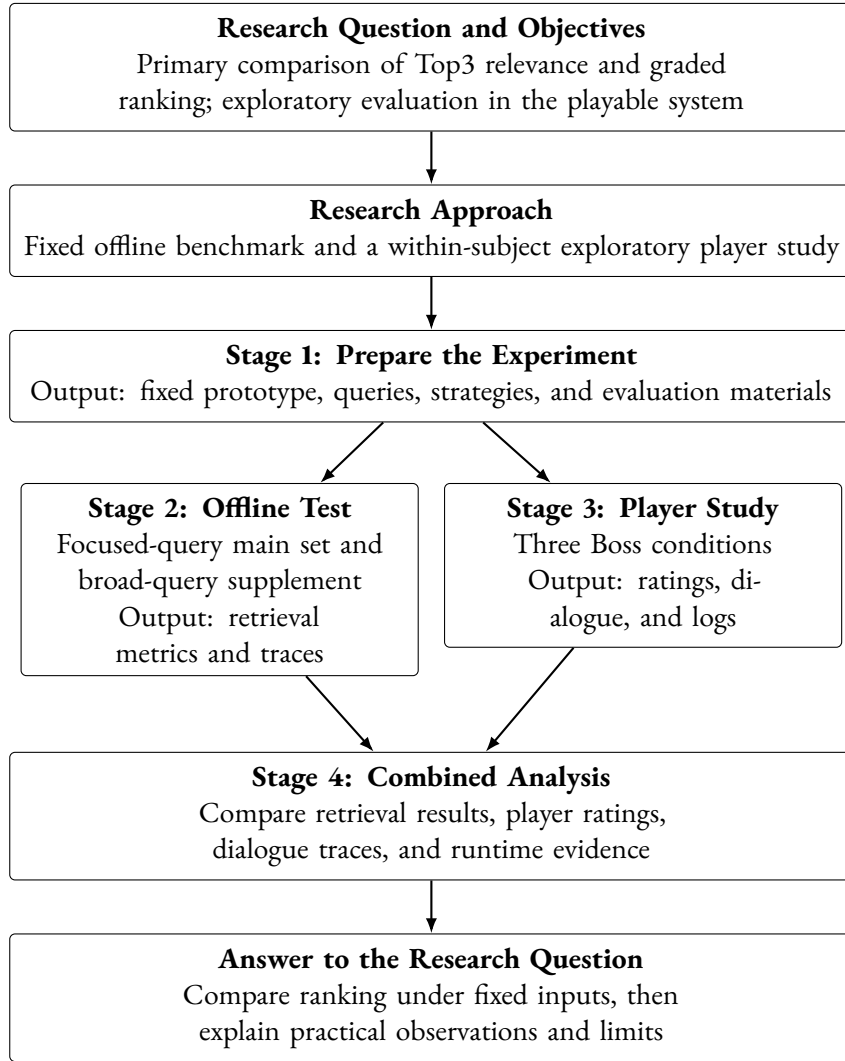


FIGURE 3.1: Overview of the research methodology and its main outputs.

while the author's existing FGR prototype had already implemented this structure. The choice therefore reduced the work needed to construct the experimental game and placed waiting at predictable interaction boundaries. It does not assume that latency disappears, or that the same wait would suit real-time action or rapid voice-dialogue games.

Besides, the queries produced by boss triggers did not all have the same scope. A focused query referred to one identifiable event or state, such as an empty-hand event or low health, whereas a broad query could require combat performance, narrative choices, and general behaviour to be considered together. Query type was not a fourth experimental condition, but it changed the scope and ease of labelling the task. Focused queries were therefore used for the primary offline comparison, while broad queries were kept as supplementary and exploratory evidence. This provides developers with more detailed references.

The player study uses a within-subject design in which every participant experiences all three conditions, making direct comparison possible in the expected sample of 8–15 participants. Three cyclic orders labelled A, B, and C place each strategy in different positions.

This cannot remove learning or fatigue completely, but avoids giving every participant the same order, which is also the easiest solution to work with Qualtrics.

3.3 STUDY PLAN AND OUTPUTS

Accordingly, the research has four stages: preparation, offline testing, player testing, and combined analysis.

3.3.1 STAGE 1: PROTOTYPE

The first phase will finalize and fix the game prototype, memory framework, query text, three strategies, evaluates materials and various basic parameters. Single and combined queries serve as the analytical boundaries for the fixed queries used in the reporting phase; they do not rewrite already recorded queries or retrieval traces. The prototype includes three mini-battles and, based on actual gameplay, forms a set of player event memories, which are then used in conjunction with the three boss conditions. Once the official collection begins, the build version will no longer be adjusted based on the results.

3.3.2 STAGE 2: OFFLINE RETRIEVAL

The second phase will eliminate the influence of dialogue generation and player actions. All three strategies will process the same 16 memories and 9 queries, returning the Top 3, which will then be compared with manually labeled results to ensure the elimination of interfering items. Four single-type queries and their 64 query-memory pairs directly answer the main RQs; the remaining five combined queries and 80 pairs are reserved for supplementary analysis.

3.3.3 STAGE 3: PLAYER STUDY

The third phase will involve Qualtrics randomly assigning participants, who are recruited from university classmates, peers or gamers, to experience three bosses in the order of A, B, or C. Participants will answer questions immediately after each boss fight to prevent forgetting. During gameplay, Unity will generate `SessionCode` values and synchronize them to Qualtrics, along with `Mode` data to reflect the specific order in which the player used the mode. The backend will generate dialogues and performance logs named after the `SessionCode` values. Because actual actions, states, triggers, and generated text will vary, this phase evaluates the entire system, rather than just a strict causal experiment that changes the retriever. Specifically, it will use player questionnaires and performance reports to measure the number of participants, percentages, mean ratings, and median latency.

3.3.4 STAGE 4: ANALYSIS

The fourth phase will summarize offline metrics, questionnaires, conversations, and performance, and then attempt to compare whether they corroborate each other. This evidence

will not be combined into a single overall score, and exploratory player ratings will not overwrite the more controlled offline results.

3.4 MEASURES OF SUCCESS

After obtaining the results from these stages, the validation of the main RQ will rely solely on single-type queries, with retrieval quality described by the metrics: Precision@3, Recall@3, and nDCG@3. Composite queries, player ratings, verifiability, and latency provide supplementary context. Among them, Precision@3 and Recall@3 primarily refer to the binary relevance of the top 3 returned content, i.e., whether it is relevant. nDCG@3 refers to whether relevant memories are ranked according to the metrics. Finally, a macro-average of the four single-type queries is used as the final result. The item-by-item results of all nine queries and the macro-average of the five composite queries will serve as supplementary results.

Questionnaire responses will be further evaluated using five-point ratings, comparative questions, and open-ended responses. The playable study did not define a minimum practically meaningful difference before data collection. Therefore, the questionnaire results are interpreted through observed mean differences, effect sizes, and participant preferences. A non-significant result indicates that this pilot did not detect a stable difference; it does not demonstrate that the three conditions were equivalent.

Questionnaire conditions were compared using the Friedman test. Kendall's W was reported as the corresponding effect size to describe how consistently participants ranked the three conditions, while p -values were treated as exploratory because of the pilot sample.

Finally, the logs will primarily analyze the inspectability data provided by the project, tracking data similarity, importance, final ranking, and error status throughout the testing process, rather than relying on external model insights. Latency will also be evaluated, categorized into preparation, online retrieval, generation, and total latency, and reported along with the number of failures.

4 METHOD

This chapter explains how the design from Chapter 3 was implemented. This includes the creation and refinement of the game prototype, gameplay design, memory architecture, retrieval strategies, dialogue generation, and data collection. Many details, such as online questionnaire, complete prompts, rating sheets, questionnaire content, and fixed configurations, are provided in the appendix.

4.1 PROTOTYPE IMPLEMENTATION

The prototype was built on FGR, an earlier Unity turn-based card-combat project created by the author [41]. Figure 4.1 shows this playable foundation. The earlier project supplied scene progression, characters and boards, card actions, turns, and battle management, as summarised in Figure 4.2. This study added gameplay-event recording, a memory pool that persists between scenes, three retrieval methods, boss-dialogue triggers, and experimental logging. Unity handled play and displayed the resulting dialogue. A local Node.js proxy ran the sentence-embedding model and forwarded dialogue-generation and model-scoring requests to the fixed LLM endpoint. Figure 4.3 shows the basic flow of the research additions rather than the full software structure of the original game.

Within this structure, Unity recorded events, the `MemorySystem` retained cross-scene memories, and a common retriever interface supplied Top3 records to the `DialogueController`.



FIGURE 4.1: Turn-based card-combat foundation inherited from the earlier FGR prototype.

The local proxy isolated external services and the application programming interface (API) key. As a result, this separation allowed retrievers to change without rewriting memory formation, triggers, or presentation.

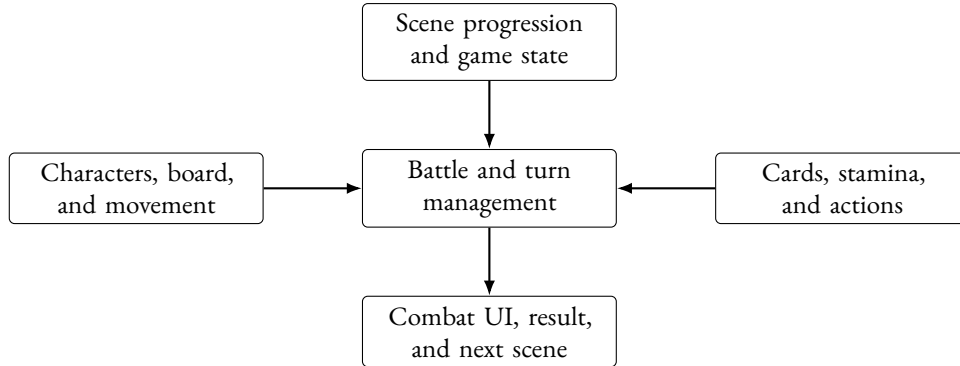


FIGURE 4.2: Main gameplay foundation inherited from the previous FGR prototype.

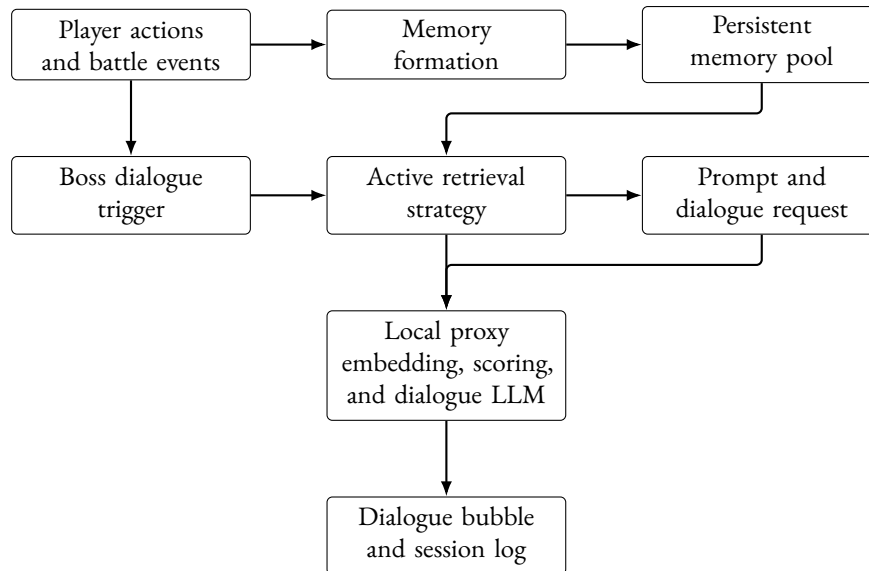


FIGURE 4.3: Flow from gameplay events through retrieval and generation to boss dialogue.

4.2 GAMEPLAY AND EXPERIMENTAL SCENARIO

The turn-based structure applied the latency consideration from the Methodology through clear trigger positions. Dialogue requests could be placed at encounter start, turn start, after a player action, or at battle end without interrupting continuous real-time control. A complete playthrough first formed memories and then entered the boss experiment. As shown in Figure 4.4, the player completed three minor battles against characters connected to Rowan through loyalty, obligation, or family, and chose how to treat each defeated opponent. The system also recorded combat, so the history came from actual play.



FIGURE 4.4: Three preliminary encounters used to form long-term episodic memories.



FIGURE 4.5: Boss encounter displaying generated dialogue in a world-space bubble.

The player then fought Rowan three times, with the dialogue presentation shown in Figure 4.5. Character, background, long-term memory, trigger rules, Top3, and generation model remained fixed; retrieval strategy and order changed. The scenario produced several memory types while keeping requests traceable. On defeat, the current boss restarted, long-term memories remained, and Working Memory was cleared.

4.3 DIALOGUE MEMORY ARCHITECTURE

Information was divided into three layers. First, Static Context contained the world, rules, Rowan’s personality, and relationships and entered every prompt. Next, Long-term Episodic Memory came from the preliminary encounters and was ranked by the three strategies. Finally, Working Memory described immediate events in the current boss attempt, entered the prompt directly, and was cleared after defeat. Full fields are listed in Appendix A.3.

Battle statistics were accumulated during play rather than reconstructed after the encounter. Damage and hand use were collected across the complete player turn and converted into one TurnEvent when that turn ended. Encounter duration, final health, and the narrative choice were added only after victory confirmed the final values. The MemorySystem persisted these records when scenes changed and assigned each record a stable identifier used by embedding caches, importance caches, and retrieval logs. Boss working memories followed a different lifecycle: they were created from events in the current attempt, inserted directly into the prompt, and removed when that attempt restarted.

The four long-term categories were NarrativeChoice, EncounterDuration, FinalHealth, and TurnEvent. Events remained atomic, while damage, hand, and health values that explained the same turn stayed together.

If preliminary encounter i contained T_i completed player turns, the long-term pool size was:

$$N_{\text{memory}} = 9 + \sum_{i=1}^3 T_i. \quad (4.1)$$

Each encounter contributed one choice, one duration, one final-health memory, and one TurnEvent per completed turn. Encounters lasting two, two, and three turns therefore produced 16 memories. Records retained their source, category, factual text, structured values, and cached vector; both interface languages used the same English retrieval text.

4.4 RETRIEVAL STRATEGIES

After memory formation, a boss trigger became a natural-language query. All strategies implemented the same interface, received the same query and long-term pool, and returned a common Top3, so later dialogue code did not need to know the ranking method. The local all-MiniLM-L6-v2-ONNX model generated and cached memory vectors before the boss stage; online retrieval generated only the query vector.

When the game entered the pre-boss stage, the MemorySystem sent the English text of every unembedded long-term memory to the local /embed endpoint. The Node.js proxy ran the local embedding model and returned one vector for each record, which Unity cached inside the corresponding MemoryRecord. During a dialogue trigger, Unity embedded only the new query, passed the query vector and memory pool to the active retriever, and supplied the returned Top3 to the DialogueController.

4.4.1 SIMILARITY-ONLY RETRIEVAL

As the simplest starting point, Similarity-only retrieval compared only how close the meaning of each memory was to the query. It used cosine similarity, where $E(q)$ is the query vector and $E(m)$ is the memory vector:

$$s_{\cos}(q, m) = \frac{E(q) \cdot E(m)}{\|E(q)\| \|E(m)\|}. \quad (4.2)$$

The system returned the three highest similarities without judging importance to Rowan, forming the simplest baseline.

4.4.2 RULE-BASED IMPORTANCE RETRIEVAL

Building on this baseline, Rule-based retrieval added an importance value from 0 to 2 that was calculated directly from structured game data. To place both parts on the same range, cosine similarity was changed from $[-1, 1]$ to $[0, 1]$, and importance was divided by two. The final rank used 70% similarity and 30% importance:

$$s_{\text{final}}(q, m) = 0.7 \left(\frac{s_{\cos}(q, m) + 1}{2} \right) + 0.3 \left(\frac{I(m)}{2} \right), \quad I(m) \in \{0, 1, 2\}. \quad (4.3)$$

The 70/30 weighting was selected as a fixed design setting rather than tuned against the relevance labels. Similarity retained the larger weight so that importance could reorder close semantic matches without replacing query relevance entirely.

Importance was calculated independently of the current query. Rules used structured combat values and the recorded consequence of each choice. The complete thresholds are shown in Table 4.1. Where conditions overlapped, the highest applicable score was used.

Memory type	Condition	Score
Duration	Two turns or fewer, or seven turns or more	2
Duration	Five to six turns	1
Duration	Three to four turns	0
Final health	Below 25%, or at least 90%	2
Final health	25% to below 50%	1
Final health	50% to below 90%	0
Turn event	At least 50% damage, or hand exhausted with at least 25% damage	2
Turn event	At least 25% damage, or hand exhausted	1
Turn event	Neither condition	0
Narrative choice	Strong, indirect, or no impact on Rowan	2, 1, or 0

TABLE 4.1: Rule-based importance thresholds.

4.4.3 MODEL-ASSISTED IMPORTANCE RETRIEVAL

The third strategy, Model-assisted retrieval, kept the same Top3, 70/30 weights, and formula but replaced the rule value with a 0–2 LLM importance score. The prompt supplied Rowan’s values, character relationships, memory text, and values and required importance to be judged independently of the current query. The full prompt appears in Appendix A.2.

Before the first boss condition, all memories without cached model scores were collected into one scoring batch. The scoring prompt supplied the rubric, Rowan’s values and relationships, and the structured facts of each record. The model returned a 0–2 score and a short reason for every `MemoryId`. Unity validated the identifiers and score range before storing valid results in a separate cache. Missing or invalid results were logged and could be retried. Consequently, normal Model-assisted retrieval reused cached values and applied the same ranking formula as Rule-based retrieval; it did not ask the model to rescore the full memory pool for every dialogue line. This separated the one-time scoring cost from cached online retrieval.

Formal dialogue generation and Model-assisted importance scoring both used the same fixed DeepSeek-V4-Pro-0813 model. The model, API endpoint, and request configuration remained unchanged across all three retrieval conditions and all participant sessions.

Model-assisted retrieval sent an additional scoring prompt to this same model rather than introducing a second LLM.

4.5 DIALOGUE GENERATION

To control when dialogue was requested, boss speech used deterministic game-state triggers rather than asking the LLM when to speak.

The implemented trigger points were:

1. Boss encounter start.
2. Boss turn start, with speech intervals alternating between one and two turns.
3. A player deals at least 25% of the Boss's maximum health as damage in a single turn.
4. A player runs out of cards in a single turn.
5. The Boss's HP first drops below 75% and 25%.
6. A player's HP first drops below 50% and 25%.
7. A player defeats a ranged or melee support unit first.
8. Boss defeat or player defeat.

Each threshold trigger was emitted only once per encounter. Normal requests shared a three-second cooldown, while the first supporting-unit defeat bypassed that cooldown because it created an immediate working memory. A battle-end request had priority over ordinary dialogue and cancelled a pending ordinary request, ensuring that the final outcome was not replaced by an earlier reaction.

After retrieval, the fixed prompt contained character and world background, current state, Working Memory, the Top3 long-term memories, and output rules. The model was instructed to use only the supplied facts and to return no more than two lines as Rowan. The full prompt appears in Appendix A.2.

The `DialogueController` then constructed the final prompt and sent it to the proxy's `/dialogue` endpoint. The proxy kept API credentials outside the Unity build and forwarded the request to the fixed external model.

Output language followed the localisation setting and appeared in a bubble above Rowan. Requests retained their trigger and supplied memories so that retrieval errors could be separated from generation errors.

4.6 EXPERIMENT PROCEDURE AND DATA COLLECTION

At the start of the player procedure, Qualtrics first presented the information and consent page and randomly assigned Mode A, B, or C. The participant entered the mode in Unity,

which ordered the strategies as shown in Table 4.2. `SessionCode` and `Mode` were retained in both the questionnaire and the log.

Mode	Boss 1	Boss 2	Boss 3
A	Similarity-only	Rule-based	Model-assisted
B	Rule-based	Model-assisted	Similarity-only
C	Model-assisted	Similarity-only	Rule-based

TABLE 4.2: Cyclic condition orders used in the experiment.

After each boss, the game paused while the participant completed that condition’s questions. The final page requested the single session JSONL file; the full questionnaire appears in Appendix A.1.

The offline retrieval test used 16 fixed representative memories and nine boss queries, producing 144 query-memory pairs. Q4, Q5, Q7, and Q8 each targeted one identifiable event or state, so their 64 pairs formed the primary offline evaluation set. Q1, Q2, Q3, Q6, and Q9 could require several kinds of player history and their 80 pairs formed the supplementary set. Each pair was labelled by hand as 0 for irrelevant, 1 for supporting relevance, or 2 for strong relevance. Without running combat or generating dialogue, all three strategies processed every query, and `Precision@3`, `Recall@3`, and `nDCG@3` were calculated from their Top3 results. The labeling guide appears in Appendix A.4.

Formal play was recorded in one JSONL session file containing session events, retrieval traces, importance-scoring results, dialogue outputs, and performance records. A shared `RequestId` connected the query, candidate and selected memories, generated response, timing, and errors for each dialogue request, while `SessionCode` and `Mode` identified the complete participant session. Retrieval traces stored each candidate’s similarity, importance, final score, rank, and selection state, while performance records separated retrieval, generation, and end-to-end time.

Finally, logs supported performance summaries and individual dialogue tracing but not a complete factual-error rate. Questionnaire data described player perception and were treated separately.

Fixed configuration appears in Appendix A.6.

5 RESULTS

This chapter reports the collected offline retrieval, participant questionnaire, generated-dialogue, and performance results. The causes of these differences and their implications for the RQ are considered in the following Analysis chapter. The current sample contains eleven participants.

5.1 DATASET

The current Qualtrics data&analysis page contained eleven responses, invalid answers are all deleted by author, such as missing one boss’s corresponding answers, unexpected game quit, or local proxy error. It contained eleven uploaded game logs. Every log recorded the Similarity-only, Rule-based Importance, and Model-assisted Importance conditions. Mode allocation was not fully balanced: two participants received Mode A, five received Mode B, and four received Mode C. In two responses, the survey’s embedded session value did not match the uploaded filename. However, the filename matched the internal SessionCode, and the assigned survey mode matched the mode stored in the log. These two responses were therefore linked through the uploaded filename and retained in the current dataset.

Eight logs contained the final SessionCompleted record and three did not. The three files without this marker still contained retrieval, dialogue, and performance records for all three strategies and had a completed matching questionnaire. They were therefore treated as missing an end marker rather than invalid questionnaire. One questionnaire value was absent: one participant did not answer Memory Relevance for the Rule-based condition. All other repeated condition ratings were present. Long-term memory pools contained between 15 and 17 records across players, reflecting differences formed through actual play rather than missing data.

Table 5.1 summarises the retained questionnaire and log dataset used by the later results.

5.2 OFFLINE RETRIEVAL EVALUATION

The offline benchmark contained 16 long-term episodic memories and nine fixed retrieval queries, giving 144 query–memory pairs. Q4, Q5, Q7, and Q8 formed 64 focused query–memory pairs for the primary offline evaluation; the other five broad queries formed 80 pairs for supplementary analysis. Every pair had a human relevance label on the predefined 0-2 scale, where 0 indicated no relevance, 1 indicated supporting relevance, and 2 indicated

Data item	Current count
Completed questionnaires	11
Uploaded logs matched to questionnaires	11
Mode A / B / C allocation	2 / 5 / 4
Logs containing all three retrieval strategies	11
Logs without a final SessionCompleted marker	3
Missing repeated-condition ratings	1 of 198
Successful dialogue outputs	221
Long-term memories per participant	15–17

TABLE 5.1: Overview of the current participant and game-log dataset.

strong relevance. Each strategy processed all nine queries, producing the expected 27 retrieval traces. All retrieval operations were successful, with no missing query, candidate memory, selected rank, or metric input.

Table 5.2 presents the primary macro-average results across the four focused queries. Similarity-only retrieval achieved the highest Precision@3 and Recall@3 at 0.917 and 0.277. Mean nDCG@3 increased from 0.569 for Similarity-only to 0.805 for Rule-based Importance and 0.860 for Model-assisted Importance. Rule-based and Model-assisted retrieval had the same mean selected label of 1.333. It is worth noting that these Recall@3 values should not be read as general retrieval failure. Several queries had more than three relevant memories, while each strategy could return only three. Therefore, the general numerical value of Recall@3 is very small, and the whole metric had a built-in ceiling below 1.

Strategy	P@3	R@3	nDCG@3	Mean selected label
Similarity-only	0.917	0.277	0.569	1.083
Rule-based Importance	0.750	0.217	0.805	1.333
Model-assisted Importance	0.750	0.222	0.860	1.333

TABLE 5.2: results from four primary focused queries (Q4, Q5, Q7, and Q8), using a fixed 16-memory pool and $K = 3$.

Within the primary focused set, each strategy selected 12 memories. Similarity-only returned 11 relevant memories, of which two were strongly relevant. Rule-based and Model-assisted retrieval each returned nine relevant memories, including seven strongly relevant items. As a supplementary result, mean nDCG@3 across the broad queries was 0.577 for Similarity-only, 0.684 for Rule-based Importance, and 0.879 for Model-assisted Importance.

Table 5.3 retains the per-query nDCG@3 values for all nine queries. Within the primary focused set, Model-assisted Importance was uniquely highest on Q7 and Q8, while Rule-

based Importance was uniquely highest on Q4 and tied with Model-assisted on Q5. Within the supplementary broad set, Model-assisted was uniquely highest on Q2 and Q9 and shared the highest value on Q1, Q3, and Q6.

Query	Short description	Role	Similarity	Rule-based	Model-assisted
Q1	Important prior actions and choices	Supplement	1.000	0.844	1.000
Q2	Previous fighting and behaviour	Supplement	0.235	0.490	1.000
Q3	Strong attacks and empty hand	Supplement	0.395	0.907	0.907
Q4	Strong attacks and performance	Primary	0.844	1.000	0.765
Q5	Empty-hand events	Primary	0.395	0.907	0.907
Q6	Tactical choices and treatment	Supplement	1.000	0.844	1.000
Q7	Danger posed to enemies	Primary	0.333	0.844	1.000
Q8	Struggle or wounded state	Primary	0.704	0.469	0.765
Q9	End-of-fight choices and performance	Supplement	0.255	0.333	0.490

TABLE 5.3: nDCG@3 by offline retrieval query using the fixed 16-memory pool and $K = 3$. Primary queries address one focused event or state; supplementary queries may use several kinds of evidence.

Figure 5.1 visualises the same per-query values and separates the primary focused set from the supplementary broad set.

5.3 QUESTIONNAIRE RESULTS

Table 5.4 maps each participant’s first, second, and third boss back to the retrieval strategy actually used and reports the mean and standard deviation for the six five-point items. The questionnaire used the response: 1 represented Strongly Agree, 2 Agree, 3 Neutral, 4 Disagree, and 5 Strongly Disagree. A lower mean therefore indicates a more positive rating. All strategy means were between 1.27 and 1.73, placing the observed ratings broadly between Strongly Agree and Agree.

Regarding each boss’s evaluations in questionnaire, Model-assisted Importance had the lowest mean for Dialogue Coherence, Current Context Awareness, and Engagement. Rule-

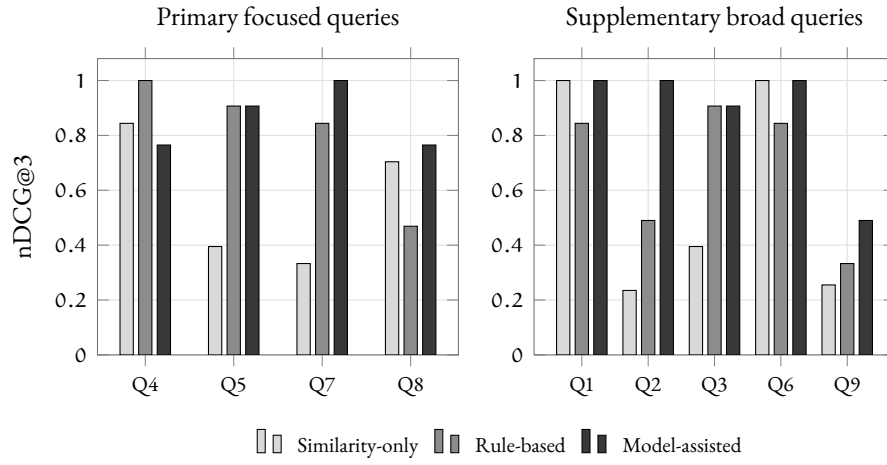


FIGURE 5.1: Queries comparison in nDCG@3. All values use the same fixed 16-memory pool and $K = 3$;

Dimension	Similarity-only	Rule-based	Model-assisted	Friedman p	Kendall's W
Dialogue coherence	1.73 (0.90)	1.64 (0.92)	1.45 (0.52)	.779	.023
Memory awareness	1.55 (0.69)	1.36 (0.67)	1.64 (0.92)	.692	.033
Memory relevance	1.36 (0.50)	1.50 (0.85) ^a	1.64 (0.67)	.549	.060
Character believability	1.55 (0.69)	1.27 (0.47)	1.36 (0.50)	.097	.212
Current context awareness	1.73 (0.79)	1.45 (1.04)	1.27 (0.47)	.311	.106
Engagement	1.45 (0.69)	1.45 (0.69)	1.27 (0.47)	.368	.091

^an = 10 because one response was missing; all other cells use n = 11.

TABLE 5.4: Post-condition questionnaire ratings by retrieval strategy, shown as mean (standard deviation), with Friedman test results and Kendall's W. Lower scores indicate more positive ratings on the 1–5 scale.

based Importance had the lowest mean for Memory Awareness and Character Believability, while Similarity-only had the lowest mean for Memory Relevance. As a descriptive summary across the six items, the overall means were 1.56 for Similarity-only, 1.45 for Rule-based Importance, and 1.44 for Model-assisted Importance. However, none of the six items met the $p < .05$ threshold. The accompanying Kendall's W values describe the observed ordering strength, ranging from .023 to .212, but are interpreted descriptively because the sample contained only eleven participants.

The comparative questions produced no majority choice. Similarity-only and Model-assisted Importance each received four overall-preference selections, while Rule-based Importance received three; the other comparative items were similarly divided. Eight participants correctly identified Rowan's intended personality. Only four questionnaires included open comments, so these responses are reported as individual observations. One participant identified the malformed line translated as "Your sword is stained with my men."; the session log linked this output to the Rule-based strategy.

5.4 LATENCY

Table 5.5 reports request-level latency for successful dialogue requests. Retrieval medians were 13.2 ms for Similarity-only, 13.9 ms for Rule-based, and 25.6 ms for Model-assisted. When each participant's median under each strategy was treated as the repeated measure, retrieval latency differed at $\chi^2(2) = 22.00$, $p < .001$, and Model-assisted had the highest median retrieval time for all eleven participants.

Strategy	Requests	Retrieval (ms)	Generation (ms)	End-to-end (ms)
Similarity-only	74	13.2	2190.6	2204.9
Rule-based Importance	73	13.9	2151.0	2165.9
Model-assisted Importance	74	25.6	1967.6	2033.1

TABLE 5.5: Median latency across successful online dialogue requests. The separate Model-assisted pre-scoring stage is excluded.

Generation and end-to-end medians were close across strategies, with no significant difference in either measure when participant medians were compared. Similarity-only contained one generation outlier of approximately 34.95 seconds, and Model-assisted contained one retrieval outlier of 7.89 seconds.

Model-assisted also required one importance-scoring step before the boss encounters. All eleven participants eventually completed this step successfully. Successful scoring took a mean of 7.39 seconds and a median of 6.88 seconds, ranging from 6.05 to 10.46 seconds. Two pre-scoring requests failed, but both were recovered by later retries. Later retrieval used the cached scores after this preparation. All 221 formal dialogue requests succeeded, giving a dialogue-request success rate of 100% in the current data.

6 ANALYSIS

Chapter 5 directly reports the offline test, player questionnaire, and runtime results. Overall, Model-assisted Importance most often placed the memories judged most useful near the front. Rule-based Importance came close while allowing the designer to see and control how importance was decided. Similarity-only found the largest number of memories that were at least related to the query, although they were not always the strongest evidence for it. The broader queries showed a similar pattern. Meanwhile, the questionnaire did not show that players could consistently notice a difference between the three strategies. This chapter therefore explains, step by step, why the offline ranking, player experience, and engineering costs do not produce exactly the same conclusion.

6.1 OFFLINE RETRIEVAL

Table 5.2 shows two different relevance patterns. Similarity-only covered more memories that were at least related to the query, whereas the two importance-aware strategies concentrated more Top3 positions on memories judged strongly relevant. Because $nDCG@3$ additionally rewards strong memories for appearing near the front, the Model-assisted result indicates an ordering advantage over Rule-based in this fixed benchmark rather than the discovery of more strong evidence.

However, the per-query results in Table 5.3 also show that no strategy was best on every focused query. Even when a query asks about one clear target, its wording and the evidence available in the memory pool can still change the ranking. Consequently, the averages are useful for summarising the overall pattern, but they do not replace checking the queries individually.

The supplementary broad-query results in Table 5.3 provide a second view and follow the same general direction, with Model-assisted ordering the graded evidence most strongly in this set. Nevertheless, a broad query may reasonably accept evidence about combat performance, player choices, or character relationships at the same time, and its labels depend more heavily on how the assessor understands the question. Accordingly, this pattern does not extend the main answer to every possible type of query.

6.2 PLAYER EXPERIENCE

The stronger offline ranking did not directly become a clearly higher player rating. As Table 5.4 shows, the most positive mean was distributed across different strategies depending on the questionnaire dimension, and the Friedman tests did not detect a consistent condition difference. The accompanying effect sizes also remain descriptive in this pilot. Put simply, the questionnaire does not provide enough evidence that one strategy was reliably rated higher, although this does not mean that the strategies behaved identically.

There are several likely reasons for this result. Firstly, every strategy usually supplied some usable information. The generation model also received the same world background, Rowan persona, current battle state, and dialogue trigger. Even when some retrieved memories were only weakly connected to the current query, this shared information was still enough for the model to produce a grammatical line in an appropriate character voice. The model could use only the easiest memory, or produce a more general boss response, without accurately using every retrieved record. Therefore, fluent dialogue did not necessarily show that the most suitable memories had been selected, and the ability of the generation model may have hidden some differences between the retrieval rankings. Secondly, the boss encounters were short, so each player triggered and read only a limited number of lines. Most of each prompt remained the same for every participant, while memories and immediate states created by individual actions formed a smaller part of the available information. Player differences therefore had fewer opportunities to change the generated dialogue, and some query and ranking differences did not appear in every playthrough. As a result, the questionnaire reflects players' overall impressions of the dialogue they encountered, whereas the offline labels examine whether individual retrieval results suited fixed queries. These sources answer two connected but different questions: whether retrieval differences were noticeable during a short play session, and whether the retriever itself selected and ordered suitable memories.

Finally, the anomalous line reported in Chapter 5 occurred during a Rule-based condition, but this association does not identify its cause. The selected memories described high-damage turns without mentioning a sword, blood, or kills, so unsupported generation or Chinese phrasing are plausible explanations. Because the outputs were not systematically coded for this error type, the example cannot support a causal, frequency, or strategy-level conclusion. It instead shows why retrieval traces and generated text need to be inspected separately when stronger factual control is required.

6.3 ENGINEERING INTERPRETATION

From an engineering perspective, the first considerations are the interpretability and replicability of the strategy. Similarity-only is the simplest, with results traceable to semantic similarity, but this score doesn't explain why an event particularly affects Rowan's sense of justice or subordinate relationships. In contrast, rule-based approaches express this

judgment as explicit thresholds, making it easier for developers to recalculate, modify, and test the results.

Model-assisted approaches replace manually enumerated importance rules with a scoring prompt, although this study did not measure the difference in development workload. Inspectability is preserved by storing an importance score and a brief reason for each memory. This approach can address narrative meanings not pre-defined by the rules, but it still depends on scoring guidance, an external API, caching, and error handling. In other words, it increases decision flexibility while introducing a different form of system maintenance.

The latency results in Table 5.5 support this interpretation. Cached Model-assisted retrieval remained short relative to dialogue generation, although its online retrieval was slower than the other strategies. This difference did not come from a new model request after the cache was complete; it mainly included local cache checks, weighted sorting, detailed trace logging, and Unity coroutine scheduling. The main strategy-specific cost was instead the separate Model-assisted pre-scoring stage reported in Chapter 5, while dialogue generation remained the main source of the player’s online wait in every condition.

The outliers reported in Chapter 5 also appeared at the beginning of affected logs. Participant support suggested that proxy startup may have contributed to these first-request delays, although the records do not establish their exact cause. They nevertheless expose startup and deployment as practical parts of the dialogue feature rather than properties of retrieval ranking alone.

The medians in Table 5.5 are more suitable for describing ordinary requests than the isolated outliers. Although Model-assisted had the lowest observed end-to-end median, this came from variation in generation rather than faster retrieval, and neither generation nor end-to-end time differed significantly. Since every condition used the same dialogue model, the result cannot be interpreted as Model-assisted accelerating generation. The shared generation wait remained noticeable for rapid real-time conversation, which supports placing requests within turn boundaries or other delay-tolerant interactions rather than assuming the same pipeline suits every game type.

The reliability results in Chapter 5 show that the formal dialogue pipeline completed the recorded requests, while Model-assisted pre-scoring also required successful retry handling. This additional API step therefore created another possible failure point. The available runs remain too limited to predict a long-term failure rate after public release.

6.4 ANSWER TO THE RQ

Within the fixed benchmark, the answer to the primary RQ is a difference in ranking behaviour rather than a universal winner. As Tables 5.2 and 5.3 show, Similarity-only emphasised coverage of at least relevant evidence, Model-assisted produced the strongest aggregate graded ordering, and Rule-based provided importance-aware ranking with more direct designer control. The per-query variation means that these aggregate tendencies should not replace testing the queries a particular game will actually use.

However, the questionnaire results further narrowed the applicability of the findings. The relatively good hierarchical ranking achieved by the model-assisted strategy did not consistently impress players; there could be many reasons for this: the quality of the generation model, the significant proportion of shared background information, shorter boss battles, and fewer actual triggered queries could all contribute to this problem. From an engineering perspective, Similarity-only provides a low-complexity starting point, Rule-based approaches use manual rule constraints to gain greater direct control, and Model-assisted approaches add pre-scoring, caching, and retry processing, increasing processing time. Therefore, these strategies are more likely to be successful only if the project's own query and memory tests can demonstrate that the ranking improvement is worth the additional preparation cost.

Finally, Table 5.5 shows that online retrieval was a small part of the shared generation wait in every condition. This latency profile is more compatible with turn-based and other delay-tolerant formats than with continuous real-time dialogue.

7 DISCUSSION

This chapter will further expand the significance of the research, provide suggestions to game developers, researchers, and point out worth-noting limitations and potential future directions.

7.1 SUMMARY OF ANALYSIS

First, Why don't differences in offline retrieval directly translate into significant differences in player experience? The retrieval engine determines which evidence enters the prompt, but the generative model can still select, ignore, or rewrite that evidence, and gameplay determines the lines of text the player actually triggers and sees. Therefore, offline ranking and player perception are not contradictory results; they describe different stages of the same process.

This gap also changes the practical meaning of the study. More complex retrieval is valuable only when stronger evidence ordering changes the final text enough to matter. In the current small memory pool and short boss encounters, every strategy usually supplied at least one usable memory, and the fixed generation model could also rely on shared context to produce fluent dialogue. The study is therefore better understood as an inspectable case of trade-offs across the pipeline, rather than proof that one retriever must produce better game dialogue.

7.2 IMPLICATIONS FOR GAME DEVELOPERS

These engineering trade-offs are especially visible for small teams because every additional model, service, and maintenance step occupies a larger share of their budget and development time. The same issues also exist in larger projects, although those teams may have more staff and infrastructure to absorb them. The following implications are therefore aimed at small projects, while still offering a useful reference for early prototyping in larger teams.

7.2.1 IMPLICATIONS FOR GAME DESIGNER

Regardless of the retrieval method, world background, character relationships, personality boundaries, and recordable events still need to be designed. Retrieval reduces the part of the history supplied to each request; it does not create that material by itself. For a small team

that wants direct control over a character’s judgement, Rule-based Importance offered a practical middle position in this prototype: it produced strong graded ranking, added little online cost, and exposed the clearest place for designers to make changes. However, the player histories contained only 15–17 long-term memories, so this study cannot show that the same choice remains best in a larger and more complicated game.

Furthermore, when designing a dialogue system, one should first determine what a character might ask or evaluate later, and then decide which events are worth remembering. For reference, this study used atomic events to better match choices, final lives, combat duration, and single-turn performance with queries. Next is practical boundary design, which should record events that the game can confirm, that a character might use later, and that can be expressed with clear fields. Finally, the retrieval system cannot recover information that the game has never recorded; therefore, memory schema and query design should precede strategy selection.

Importance scores themselves also stem from character design. In this project, the same action might have completely different meanings for Rowan and another boss. Rule-based systems require developers to directly maintain these judgments, while model-assisted systems write criteria into the scoring prompt and still require sample verification. Model-assisted systems change the form of design and verification work but do not eliminate this work; therefore, as the number of characters and events increases, the team still needs to invest some manual maintenance costs for rules, scoring criteria, or validation samples.

7.2.2 IMPLICATIONS FOR GAME ENGINEERING

Integrating LLM into games adds deployment issues not present in traditional dialogue systems, including memory management, retrieval, model interfaces, request generation, and failure handling. This study required testers to start a local proxy to communicate with a fixed LLM, and indeed, issues arose such as participants forgetting to start the proxy or misconfiguring it. Production products should automate these steps for players as much as possible, but simply packaging readable API keys into the client is insufficient and unsafe. Developers need to choose between self-hosted services, player-configured systems, local models, and commercial interfaces, considering security, cost, and ease of use.

Separating memory formation, retrieval, generation, and presentation allowed the three retrievers to be exchanged without changing boss triggers or the generator. A shared `RequestId` linked each query, `Top3`, generated line, and timing, while caching, cooldown, and battle-end priority controlled repeated work and request conflicts. These mechanisms did not improve offline metrics, but they determined whether the system could be inspected and run reliably during play.

The runtime results further show that developers should not compare retrieval milliseconds alone. Online retrieval under all three strategies was much shorter than the shared dialogue-generation wait, while the main Model-assisted cost appeared in preparation and an additional failure path. Similarly, problems caused when participants did not start the proxy belonged to deployment rather than ranking. The model, proxy, caches, request

scheduling, and logs are therefore all part of the delivered feature. These findings are brought together as conditional developer guidance in Table 7.1.

Project situation	Suggested starting point	Reason and caution
Early prototype or small memory pool	Similarity-only	Few strategy-specific parts and useful binary relevance; character importance remains indirect
Stable mechanics and clear character values	Rule-based Importance	Direct control, reproducible scores, and low online cost; rules must be maintained as content changes
Broad questions or event meanings that are difficult to enumerate	Model-assisted Importance	Showed the clearest broad-query ordering advantage here; depends more on scoring guidance, validation, caching, and another failure path
Unclear or changing query needs	Build a small benchmark first	Query wording and available evidence may matter more than adding a more complex ranker

TABLE 7.1: Conditional guidance for choosing a retrieval strategy in a small game project.

7.3 IMPLICATIONS FOR RESEARCHERS

The unusual Chinese wording reported in the open feedback also raises a question beyond retrieval: even when the supplied memory is correct, the generation model may extend it in an unsupported way. The current trace links this case to the Rule-based condition, but the other outputs were not exhaustively coded and the same request was not generated in English. It is therefore not possible to determine whether the problem came from retrieval, Chinese wording, or the model itself. Later work could separately compare stricter grounding constraints, direct Chinese generation, and an English-first translation pipeline rather than turning one case into a general error rate.

This study also fixed the ratios between player long-term memory, static context, model, and TopK, which could be a future research direction. The world and character background in the current prompt remain unchanged for each request, while only the top 3 player memories are provided each time, potentially mitigating the differences caused by different player histories. The project’s source code [42] also retains a full-memory mode, which can be used for future research comparing longer histories, different models, and different TopK. Such research can explore whether TopK should be a fixed value, a ratio that varies with the memory pool, or determined by both model context and the number of tasks, rather than assuming 3 applies to all games.

Finally, the author further analysed the focused and broad query results reported in Table 5.3, although query scope was not an additional experimental condition. No strategy led every focused query, whereas Model-assisted showed the clearest ordering pattern across the broad set. One possible explanation is that broad questions require several kinds of

information, such as player choices, combat performance, and character relationships, to be considered together. Model-assisted scoring can interpret relationships between these sources, while fixed rules and semantic similarity may not cover every combination. However, broad questions also permit more than one reasonable answer, so both human labels and model scores depend more heavily on interpretation. This pattern is therefore a direction for later testing rather than a general conclusion about query types.

7.4 LIMITATIONS

7.4.1 TEST ENVIRONMENT AND PARTICIPANTS

The game prototype was not an ideal lightweight test baseline. One intended participant using a GTX 1650 could not run it reliably, showing that shaders and rendering requirements unrelated to retrieval raised the participation threshold and could mix graphics problems with dialogue ratings. The retained sample and uneven mode allocation are reported in Table 5.1. Because the three sequence groups were not evenly represented, learning, fatigue, and encounter-order effects cannot be completely ruled out and may have influenced the questionnaire means. Qualtrics randomisation did not guarantee balanced retained groups because the multi-step startup and upload process also produced invalid retries. Consequently, the data can show feasibility and early patterns but cannot establish a general player preference. A lighter client, low-quality mode, and simpler study procedure would reduce these limitations.

7.4.2 CONTROL OF THE PLAYABLE STUDY

Retrieval strategy was not the only changing factor in the playable study. Query scope and available evidence differed, player actions changed Working Memory, trigger frequency, and dialogue count, and outputs from the same fixed generation model were not fully deterministic. Player ratings therefore cannot be attributed only to the retriever and should instead be treated as an exploratory evaluation of the complete system. This does not indicate variation in the LLM: every condition and participant session used DeepSeek-V4-Pro-0813. A later experiment would need a more consistent trigger set and a longer but controlled player history to separate retrieval effects more clearly from player behaviour and generation variation.

7.4.3 TECHNICAL GENERALISABILITY

The generalisability of the offline and engineering results was jointly limited by the fixed test material, technical configuration, and prototype type. As described in Chapters 3 and 5, the benchmark used a small fixed query set, one memory pool, one assessor, and one boss persona, Rowan.

One of the most noteworthy aspects is, all relevance labels were assigned by the system designer, who was familiar with the rule thresholds, Rowan's intended values, and the

expected behaviour of the three strategies. Relevance and importance were defined as different judgements, so the labels did not directly copy the rule-based importance scores. Nevertheless, prior knowledge of the system may have influenced ambiguous decisions, particularly where several memories provided reasonable support for a broad query. This may have favoured memories that matched the intended rule and model-assisted definitions, thereby affecting Precision@3, Recall@3, and nDCG@3. Ideally, one or more independent assessors should apply the same written guide without seeing strategy names, importance scores, or retrieval rankings.

Besides, every condition also used the same DeepSeek-V4-Pro-0813 model for dialogue and scoring and the same local embedding model. This fixed setup improved consistency and made the comparison recalculable, but it cannot show that other query phrasings, player histories, characters, assessors, or models would produce the same result. The internal execution of the external API could not be inspected, and network or provider conditions could still affect latency. In addition, turn-based combat could place model requests within natural pauses, so the effect of an approximately two-second wait may be smaller here than in real-time action, multiplayer communication, or continuous voice dialogue. The study also did not conduct long-duration load testing or measure the labour needed to maintain rules, scoring prompts, or the proxy. The findings therefore describe this fixed system rather than universal performance across models and game types or a measured saving in development time.

7.5 FUTURE WORK

If this research continues, the author believe the top priorities should be increasing the number of independent annotators, expanding the proportion of player events in the prompt, and increasing the player sample size. These three aspects will respectively strengthen the offline reference standard, the chance of strategies producing visible differences, and the stability of questionnaire results, thus allowing the current research findings to support broader scenarios.

Specifically, subsequent offline research should first expand the query set, adding multiple query types to each event, and fixing the classification rules for single and combined queries for further comparison. Then, multiple player memory pools and multiple independent annotators should be tested to check if the ranking trend is stable. Relevance thresholds, allowing fewer than Top 3 returns, and different Top K value should also be tested separately; a larger memory pool can also be used to compare when retrieval and existing full-memory patterns begin to produce real differences.

Furthermore, subsequent player research also needs a larger sample size and the selection of more reliable random assignments to balance the order. Longer player histories can increase the chance of strategies producing different results, while retaining a set of common trigger points encountered by all participants, making retriever effects easier to distinguish from differences in player behavior. The questionnaire was completed immediately after

each Boss battle, but the startup and log upload steps need further simplification.

Engineering optimizations should include using a lighter client, defaulting to low-quality mode, an automated proxy deployment process, and visual inspection tools. Stress testing should be conducted to check for issues such as continuous triggering, timeouts, scoring failures, and cache invalidation. The generated text also requires more specialized factual encoding and stricter constraints. A direct Chinese generation process versus a localization process that generates English first and then translates it can also be compared separately.

8 CONCLUSION

8.1 STUDY SUMMARY

This study addresses a practical problem for small-scale game developers: when a boss character generates dialogue based on player behavior, how should the game control and select the most relevant memories from player behavior history to provide to a large language model, thereby making the entire system more controllable, understandable, high-performing, and cost-effective? The study implemented three alternative retrieval strategies—Similarity-only, Rule-based Importance, and Model-assisted Importance—in a Unity turn-based card game prototype, and compared them under the same conditions of fixed character personalities, prompt structure, Top 3 size, and fixed generation model.

The study used two layers of evidence. Fixed offline benchmarks examined memory selection and ranking, while playable experiments and logs observed the entire process. Therefore, retrieval quality, player experience, verifiability, and runtime cost were evaluated separately. A simple conclusion can be drawn: better retrieval ranking does not guarantee that the generation model will use these memories, nor does it guarantee that players will perceive the differences.

8.2 MAIN FINDINGS

First, the offline results reported in Chapter 5 show that the strategies emphasised different aspects of retrieval. Similarity-only provided broader coverage of at least relevant memories, the importance-aware strategies concentrated more strongly on highly relevant evidence, Model-assisted produced the strongest graded ordering in the aggregate result, and Rule-based offered the most direct and easily recalculated importance control. The answer to the main RQ is therefore a trade-off between coverage, strong-evidence ordering, and designer control rather than a simple ranking.

The supplementary broad-query pattern in Table 5.3 gives Model-assisted scoring more reason for later investigation. Questions that accept several information types create more room for importance scoring, but they also increase the interpretation involved in human labels and model decisions. Since the study used one fixed model and query set, this remains a hypothesis for later cross-model research.

Then, the player experiment did not produce the same clear quality ranking as the offline test. All three methods received relatively positive reviews, with no questionnaire items

showing significant differences or forming majority preferences. Factors such as shorter boss battles, smaller memory pools, different triggers, and fixed generation models may have mitigated the differences. Therefore, the benefits of offline sorting did not consistently translate into perceptible improvements for players.

In terms of performance, Table 5.5 shows that online retrieval was a small part of the shared generation wait. Similarity-only required the fewest strategy-specific components, Rule-based required maintaining explicit rules, and Model-assisted added a separate pre-scoring stage together with verification and retry handling. This latency profile is more suitable for turn-based and other delay-tolerant game formats than for continuous real-time dialogue.

8.3 CONTRIBUTIONS

This study does not propose a new general RAG algorithm, but instead provides a reproducible game-specific LLM dialogue framework and a runnable, traceable end-to-end case. Three strategies handle the same memory, query, TopK, and human labels; the baseline connects the implementations of game events, three-layer memory, replaceable retrievers, proxies, caching, request scheduling, and RequestId logging. Two-layer evaluation demonstrates that retrieval ranking is relevant to but not substituted for the final dialogue experience, and also shows how small projects can organize the recording and failure handling around the generation model.

For small game development groups, the final choice of retrieval method should start from what the character would actually ask, the events the game can reliably record, and the complexity the team is willing to maintain. Similarity-only is suitable as a starting point for low-complexity testing; rule-based approaches offer a more reliable solution when further control is needed over mechanisms and character value that the game needs to express; and model-assisted importance may be worth considering when event combinations are difficult to enumerate, although the current pilot does not establish it as the best option for larger games. Comprehensive queries may give more room for model-assisted scoring, but this hypothesis needs to be validated across models and human labels.

Finally, these factors need to be considered when introducing LLM dialogue generation into actual games. Better sorting doesn't necessarily improve the player experience immediately, and lower online latency doesn't equate to easier maintenance. Retrieval quality, final text, player experience, verifiability, and engineering costs should be recorded and tracked separately, and then comprehensively evaluated based on the project. The main value of this prototype is demonstrating how developers can use traceable testing to understand what each layer truly changes before adding complexity. The practical outcome is therefore a conditional choice rather than a universal winner, as summarised in Table 7.1.

APPENDICES

A.1 PARTICIPANT INFORMATION, CONSENT, AND QUESTIONNAIRE

The deployed online questionnaire is available through Qualtrics [43].

A.1.1 PARTICIPANT INFORMATION AND CONSENT

The opening page explains that the study evaluates dialogue produced by a memory-based boss NPC in a turn-based game.

Before continuing, participants confirm all the following statements:

1. I confirm that I am at least 18 years old.
2. I have read and understood the participant information provided above.
3. I understand what gameplay, questionnaire, generated-text, and technical-log data will be collected.
4. I understand that participation is voluntary and that I may stop before submission without penalty.
5. I consent to take part in this study.

A.1.2 QUALTRICS PRESENTATION FLOW

Table A.1 summarises how the survey and Unity build divided the participant procedure.

The Qualtrics survey stores the assigned mode and order as embedded data. The fields used by the current design are `ModeAssigned`, `ConditionOrder`, `session`, `mode`, and `encounter`. Embedded fields support routing and later linkage; they are not questionnaire outcomes. The shared `SessionCode` links a submitted survey to the participant’s combined JSONL without collecting a name. The exported Qualtrics data should retain the actual strategy represented by each condition position so that ratings are analysed by Similarity-only, Rule-based Importance, and Model-assisted Importance rather than by “Boss 1”, “Boss 2”, and “Boss 3” alone.

Stage	Qualtrics activity	Unity activity
1	Display information and collect consent.	Game not yet started.
2	Randomizer assigns A, B, or C and stores the condition order.	Participant enters the displayed mode code.
3	Survey remains open on the first condition page.	Participant completes the three memory-forming encounters.
4	Participant completes the repeated post-condition block after each boss.	Unity pauses progression after each boss until the participant continues.
5	Comparative, personality, confidence, and open questions are displayed.	Experiment has finished.
6	Participant uploads the combined session JSONL and submits the response.	No further game action is required.

TABLE A.1: Qualtrics and Unity participant flow.

A.1.3 REPEATED POST-CONDITION QUESTIONNAIRE

The following six items are shown after each boss condition. Responses are coded as: 1 = Strongly Agree, 2 = Agree, 3 = Neutral, 4 = Disagree, and 5 = Strongly Disagree. This is the same direction used in the Results chapter, so lower values indicate more positive ratings.

1. **Dialogue Coherence:** The boss's dialogue was logically consistent with the immediate exchange.
2. **Memory Awareness:** The boss appeared to remember my earlier actions or choices.
3. **Memory Relevance:** The past events mentioned by the boss were relevant to the current encounter.
4. **Character Believability:** The boss presented a consistent and recognisable personality.
5. **Current Context Awareness:** The boss appeared aware of the current battle phase, character status, and immediate situation.
6. **Engagement:** The boss's dialogue increased my immersion in the encounter.

A.1.4 PERSONALITY, COMPARATIVE, AND OPEN QUESTIONS

Personality recognition is measured with the question: *Which description best represents the boss's personality?* The response options are:

1. (A) Principled, formal, and constrained by rigid ideas of justice;

2. (B) Cruel, impulsive, and primarily motivated by destruction;
3. (C) Opportunistic, persuasive, and motivated by personal gain;
4. (D) Unsure.

After all three conditions, participants answer the following comparative questions:

1. Which version used your previous actions most effectively?
2. Which version mentioned the most irrelevant, forced, or repetitive information?
3. Which version presented the clearest boss personality?
4. Which version did you prefer overall?
5. Did any version forget, misunderstand, alter, or invent a previous event?
6. How confident are you in these judgements?
7. Do you have any suggestions for the game or dialogue content?

The first four comparative items should identify versions using the neutral condition labels shown to the participant. The analysis then converts those positions to their actual retrieval strategies using the stored mode and condition order. The final question is an open-text item; the question about memory errors should include an optional explanation field so that a reported problem can be compared with the corresponding dialogue output.

A.2 DIALOGUE AND IMPORTANCE-SCORING PROMPT SPECIFICATION

A.2.1 DIALOGUE-GENERATION PROMPT

Each request supplies the following blocks in order: role, world background, game background, Rowan's persona, the identities and relationships of the three minor enemies, current scene and trigger, current-attempt working memory, and up to three retrieved long-term episodic memories. The role identifies Rowan Serenade as the boss speaking directly to the player knight. The background explains the kingdom, oppressed villages, Rowan's aristocratic upbringing and conflict with the royal family, the player's mission, and Rowan's relationship with each defeated subordinate.

Runtime fields include SceneId, Intent, immediate-situation description, output language, working-memory statements, and selected memory records. Each retrieved item is formatted with its battle identifier, memory category, and factual text.

```

public class DialogueGenerator
{
    private const string WorldBackground =
        "The story takes place around a kingdom ruled by a royal family. " +
        "Rowan's forces guard a nearby snow mountain, while people in the surrounding " +
        "villages live under oppression connected to the kingdom's ruling order.";

    private const string GameBackground =
        "The player is a talented knight who became well known through the kingdom's " +
        "Knight Selection. The Emperor sent the player and a team to eliminate Rowan, " +
        "whom the kingdom describes as a nearby villain. The rest of the team died on " +
        "the journey, and the player reached Rowan alone after three minion encounters.";

    private const string BossPersona =
        "Rowan Serenade was raised in a wealthy aristocratic family and received a rich " +
        "education and formal combat training. After the family was ruined and former " +
        "allies turned against them, Rowan came to hate the royal family and the kingdom. " +
        "When persecuted, Rowan was rescued by people from nearby villages and witnessed " +
        "their oppressed lives. Rowan is chivalrous and strongly committed to a rigid idea " +
        "of justice, seeks revenge for past wrongs, and intends to destroy the kingdom's " +
        "ruling order. Rowan does not consider killing the player necessary and initially " +
        "sees the player as another knight deceived by the Emperor, so Rowan may try to " +
        "persuade the player and reveal the truth. However, Rowan also resents the player " +
        "for harming Rowan's forces. Rowan speaks with aristocratic dignity: formal, " +
        "controlled, concise, admonitory, and emotionally restrained.";

    private const string MinionBackground =
        "Battle1 - Loyal Robot Guard: Rowan's inorganic creation guarded the snow mountain. " +
        "It had no emotions but was completely loyal to Rowan.\n" +
        "Battle2 - Captured and Coerced Ally: Rowan protected this guard's family, so the " +
        "guard reluctantly helped defend the snow mountain.\n" +
        "Battle3 - Rowan's Kinsman and Subordinate: This guard shared Rowan's background, " +
        "accepted Rowan's goals, and willingly assisted Rowan.";

    private string proxyUrl = "http://127.0.0.1:3000/dialogue";
}

```

FIGURE A.1: predefined prompts example.

A.3 MEMORY SCHEMA

Static context, long-term episodic memory, and working memory have different roles. Static context contains the fixed world, game mechanics, Rowan persona, and subordinate relationships and always enters the dialogue prompt. Long-term episodic memory contains atomic records generated from the player's first three encounters and is the only layer ranked by the three retrieval strategies. Working memory contains immediate facts from the current boss attempt, enters the prompt directly, and is cleared when that attempt is restarted. Table A.2 lists the structured fields retained for each long-term category.

Category	Stored fields
NarrativeChoice	BattleId, factual choice text, ChoiceIndex, NarrativeConsequence
EncounterDuration	BattleId, factual duration text, TurnCount
FinalHealth	BattleId, factual health text, RemainingHealthPercent
TurnEvent	BattleId, TurnIndex, TurnDamage, TurnDamagePercent, HandExhausted, PlayerHealthPercentAfterTurn

TABLE A.2: Long-term episodic memory categories and structured fields.

Every stored record also receives a unique memory identifier and a semantic vector before

retrieval. Importance is not written into the record as permanent truth: rule importance is calculated from the structured fields, while model importance is stored in a separate cache with its returned reason and error state.

A.4 HUMAN RELEVANCE LABELING GUIDE

The offline benchmark judged how useful each memory was for its current query, rather than its general narrative importance or whether Rowan approved of the event. Shared words alone did not make a memory relevant. The same guide was applied to all 144 query–memory pairs:

Label	Meaning	Decision boundary
0	Irrelevant	Does not help answer the query; a shared battle, character, or keyword is insufficient by itself.
1	Supporting	Provides related or indirect evidence, but does not directly identify the requested action, event, or state.
2	Strong	Directly answers the query or provides one of its clearest available pieces of evidence.

TABLE A.3: Minimal human relevance guide used for the offline benchmark.

Broad queries could assign label 2 to more than one memory when several records directly supported the requested judgement. One assessor applied the guide to the fixed set, so the labels are treated as reference judgements rather than absolute ground truth. They were used only to evaluate retrieval outputs and were not supplied to the three retrievers. The complete matrix is provided in `Relevance_Labels_144.xlsx` with the project materials.

A.5 EXPERIMENTAL CONDITION ORDERS

Mode	Boss condition 1	Boss condition 2	Boss condition 3
A	Similarity-only	Rule-based	Model-assisted
B	Rule-based	Model-assisted	Similarity-only
C	Model-assisted	Similarity-only	Rule-based

TABLE A.4: Three cyclic experimental condition orders.

Qualtrics assigns the mode before the boss encounters. The participant enters A, B, or C in Unity, and the game resolves each boss condition using Table A.4. A random six-character `SessionCode` links the survey and combined session log. These three cyclic orders balance first, second, and third positions across strategies, although they do not represent all six possible permutations.

A.6 RELEASE, PROXY CONFIGURATION, AND REPRODUCIBILITY

The source and release archive are available from the [42]. The manual lable file exists in root folder as well. The release contains the Windows game and a portable LLMProxy folder. `start.bat` launches the bundled Windows x64 runtime, preloads the local embedding model, and listens on `127.0.0.1:3000`, so a separate Node.js installation is not required. During formal data collection, the study fixed DeepSeek-V4-Pro-0813, the provider endpoint, and the request configuration for every participant and condition; participants did not select an LLM. The public repository does not include the private API URL or key. A future replicator must therefore supply credentials in a local `.env`; using a different model would be an extension of the implementation rather than an exact reproduction of the reported experiment. Table A.5 summarises the fixed configuration used for the reported runs.

Component	Fixed configuration
Unity	2022.3.62f2
Dialogue and scorer model	DeepSeek-V4-Pro-0813 for every condition and session; both tasks used the same fixed model
Embedding model	onnx-community/all-MiniLM-L6-v2-ONNX, fp32 CPU inference
Proxy endpoints	POST /dialogue, POST /embed, and GET /health on localhost port 3000
Retrieval	TopK = 3; normalised cosine similarity; importance scores 0–2
Fusion	Similarity-only: 1.0S; Rule-based and Model-assisted: $0.7S + 0.3I$
API request fields	Model identifier, user message, <code>thinking.type=disabled</code> , and <code>reasoning_effort=none</code>
Omitted API fields	Temperature and maximum output tokens use the unchanged provider defaults
Output rule	English or Simplified Chinese; no more than two short sentences

TABLE A.5: Fixed implementation configuration.

Each participant run produces one combined JSONL with four record types:

- `MemoryRetrievalTrace` supports retrieval inspection;
- `MemoryImportanceScoring` supports model-score inspection;
- `DialogueOutput` supplies generated text for dialogue coding; and
- `DialoguePerformance` supports latency and reliability analysis.

The shared `RequestId` connects records describing the same generation request, while `SessionCode` and `mode` connect the game file to the Qualtrics response.

BIBLIOGRAPHY

1. Riedl, M. O. & Bulitko, V. Interactive Narrative: An Intelligent Systems Approach. *AI Magazine* **34**, 67–77. doi:10.1609/aimag.v34i1.2449 (2013).
2. Mateas, M. & Stern, A. Structuring content in the Façade interactive drama architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* **1**, 93–98 (2005).
3. Chen, S., Nelson, M. & Mateas, M. Evaluating the authorial leverage of drama management. In *Proceedings of the AAAI conference on artificial intelligence and interactive digital entertainment* **5**, 136–141 (2009).
4. Vaswani, A. *et al.* Attention Is All You Need. In *Advances in Neural Information Processing Systems* **30** (2017). <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
5. Brown, T. B. *et al.* Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems* **33**, 1877–1901 (2020). <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
6. Burnes, A. *NVIDIA ACE Autonomous Game Characters Debut in inZOI and NARAKA: BLADEPOINT MOBILE PC VERSION* Reports on-device ACE characters in released game experiences. NVIDIA GeForce News. <https://www.nvidia.com/en-eu/geforce/news/nvidia-ace-naraka-bladepoint-inzoi-launch-this-month/> (2026).
7. Vasquez, A., Nnoli, I. & Spaulding, J. *Bring NVIDIA ACE AI Characters to Games with the New In-Game Inferencing SDK* Technical architecture for real-time speech, language inference, retrieval, voice, animation, and GPU scheduling. NVIDIA Developer Technical Blog. <https://developer.nvidia.com/blog/bring-nvidia-ace-ai-characters-to-games-with-the-new-in-game-inference-sdk/> (2026).
8. Inworld AI. *How Inworld Helped Death by AI Reach Profitability at 20M Players* Industry case study discussing inference cost, latency, scalability, model quality, and localisation. Inworld AI. <https://inworld.ai/blog/how-inworld-helped-the-ai-game-death-by-ai-with-20-million-players-reach-profitability> (2026).

9. Hejmadi, K. *Augmenting NPCs with LLMs: Practical Experience from NC State University & Arm Developer Labs* Conference session presented by Arm. Game Developers Conference. <https://gdcvault.com/play/1035613/Augmenting-NPCs-with-LLMs-Practical> (2026).
10. Kjun. *Bringing More Life to inZOI* Developer update discussing Smart Zoi response failures and the discontinuation of an AI feature that did not meet expected quality. inZOI Forums. <https://forum.playinzo.com/t/topic/15348> (2026).
11. Liu, N. F. *et al.* Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* **12**, 157–173 (2024).
12. Li, Z., Li, C., Zhang, M., Mei, Q. & Bendersky, M. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* 881–893 (2024).
13. Lewis, P. *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* **33**, 9459–9474 (2020).
14. Park, J. S. *et al.* Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology* 1–22 (2023).
15. Gao, Y. *et al.* *Retrieval-Augmented Generation for Large Language Models: A Survey* 2023. doi:10.48550/arXiv.2312.10997. arXiv: 2312.10997 [cs.CL].
16. Akoury, N., Yang, Q. & Iyyer, M. A framework for exploring player perceptions of llm-generated dialogue in commercial video games. In *Findings of the Association for Computational Linguistics: EMNLP 2023* 2295–2311 (2023).
17. Peng, X. *et al.* Player-driven emergence in llm-driven game narrative. In *2024 IEEE Conference on Games (CoG)* 1–8 (2024).
18. Buongiorno, S., Klinkert, L., Zhuang, Z., Chawla, T. & Clark, C. PANGeA: procedural artificial narrative using generative AI for turn-based, role-playing video games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* **20**, 156–166 (2024).
19. Yu, H. & Riedl, M. Data-driven personalized drama management. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment* **9**, 191–197 (2013).
20. Sadeq, N. *et al.* Mitigating hallucination in fictional character role-play. In *Findings of the Association for Computational Linguistics: EMNLP 2024* 14467–14479 (2024).
21. Tseng, Y.-M. *et al.* Two Tales of Persona in LLMs: A Survey of Role-Playing and Personalization. In *Findings of the Association for Computational Linguistics: EMNLP 2024* 16612–16631 (Association for Computational Linguistics, 2024). doi:10.18653/v1/2024.findings-emnlp.969.

22. Baddeley, A. D. & Hitch, G. J. in *Psychology of Learning and Motivation* (ed Bower, G. H.) 47–89 (Academic Press, 1974). doi:10.1016/S0079-7421(08)60452-1.
23. Tulving, E. in *Organization of Memory* (eds Tulving, E. & Donaldson, W.) 381–403 (Academic Press, New York, 1972).
24. Summers, T. R., Yao, S., Narasimhan, K. & Griffiths, T. L. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427* (2023).
25. Packer, C. *et al.* Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560* (2023).
26. Wu, D. *et al.* Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813* (2024).
27. Guu, K., Lee, K., Tung, Z., Pasupat, P. & Chang, M.-W. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning* **119**, 3929–3938 (PMLR, 2020). <https://proceedings.mlr.press/v119/guu20a.html>.
28. Borgeaud, S. *et al.* Improving Language Models by Retrieving from Trillions of Tokens. In *Proceedings of the 39th International Conference on Machine Learning* **162**, 2206–2240 (PMLR, 2022). <https://proceedings.mlr.press/v162/borgeaud22a.html>.
29. Salton, G., Wong, A. & Yang, C.-S. A Vector Space Model for Automatic Indexing. *Communications of the ACM* **18**, 613–620. doi:10.1145/361219.361220 (1975).
30. Reimers, N. & Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)* 3982–3992 (2019).
31. Karpukhin, V. *et al.* Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* 6769–6781 (2020).
32. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A. & Gurevych, I. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663* (2021).
33. Farrell, R., Ware, S. G. & Baker, L. J. Manipulating narrative salience in interactive stories using Indexer’s Pairwise Event Salience Hypothesis. *IEEE Transactions on Games* **12**, 74–85 (2019).
34. Xu, W. *et al.* A-MEM: Agentic Memory for LLM Agents 2025. arXiv: 2502.12110 [cs.CL]. <https://arxiv.org/abs/2502.12110>.
35. Sun, W. *et al.* Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 conference on empirical methods in natural language processing* 14918–14937 (2023).

36. Kent, A., Berry, M. M., Luehrs Fred U., J. & Perry, J. W. Machine Literature Searching VIII. Operational Criteria for Designing Information Retrieval Systems. *American Documentation* **6**, 93–101. doi:10.1002/asi.5090060209 (1955).
37. Kekäläinen, J. & Järvelin, K. Using graded relevance assessments in IR evaluation. *Journal of the American Society for Information Science and Technology* **53**, 1120–1129 (2002).
38. Järvelin, K. & Kekäläinen, J. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)* **20**, 422–446 (2002).
39. McDonnell, T., Lease, M., Kutlu, M. & Elsayed, T. Why is that relevant? collecting annotator rationales for relevance judgments. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing* **4**, 139–148 (2016).
40. Saad-Falcon, J., Khattab, O., Potts, C. & Zaharia, M. Ares: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* 338–354 (2024).
41. Yang, S. *FGR* <https://github.com/GEShunyYang/FGR>. Online; accessed 20-AUG-2026. 2026.
42. Yang, S. *BOSSNPC* <https://github.com/GEShunyYang/BossNPC>. Online; accessed 20-Aug-2026. 2026.
43. Yang, S. *Player-Study Questionnaire for Memory Retrieval in LLM-Driven Boss Dialogue* Qualtrics survey instrument. University of Warwick. https://warwick.co1.qualtrics.com/jfe/form/SV_0dojd6cAIH5rami (2026).